🔐 Security Review — nox-protocol-contracts

Scope

Mode	default
Files reviewed	`contracts/NoxCompute.sol` · `contracts/shared/TypeUtils.sol` · `contracts/shared/HandleUtils.sol` `contracts/sdk/Nox.sol`
Confidence threshold (1-100)	80
Date	2026-05-04
Auditor	pashov-style multi-agent AI audit (8 parallel agents)

Findings

85 1. Transient ACL holder can permanently mark handle publicly decryptable

`NoxCompute.allowPublicDecryption` · Confidence: 85

Description

`allowPublicDecryption` is gated only by `onlyAllowed`, which `_isAllowedTransient` satisfies, so any contract that legitimately receives short-lived transient access can flip `$.isPubliclyDecryptable[handle]=true` with no inverse function — turning a one-tx delegation into permanent worldwide plaintext exposure of the encrypted value.

Fix

```
+ modifier onlyPersistentAdmin(bytes32 handle) {
+     NoxComputeStorage storage $ = _getNoxComputeStorage();
+     require($.admins[handle][msg.sender], UnauthorizedSender(msg.sender));
+     _;
+ }

- function allowPublicDecryption(bytes32 handle) external override
notPublicHandle(handle) onlyAllowed(handle) {
+ function allowPublicDecryption(bytes32 handle) external override
notPublicHandle(handle) onlyPersistentAdmin(handle) {
      NoxComputeStorage storage $ = _getNoxComputeStorage();
      $.isPubliclyDecryptable[handle] = true;
      emit AllowedPublicDecryption(msg.sender, handle);
  }
+ function disallowPublicDecryption(bytes32 handle) external
notPublicHandle(handle) onlyPersistentAdmin(handle) {
+     _getNoxComputeStorage().isPubliclyDecryptable[handle] = false;
```

```
+     emit DisallowedPublicDecryption(msg.sender, handle);
+ }
```

---

82 2. Transient ACL holder can mint persistent admins for third parties

`NoxCompute.allow` · Confidence: 82

Description

`allow` is gated by `onlyAllowed`, which transient access satisfies, so a contract holding only transient rights can write `admins[handle][anyAddress]=true` for arbitrary third parties; combined with the absence of any `removeAdmin`/inverse setter, the grantor cannot undo the escalation.

Fix

```
- function allow(bytes32 handle, address account) external override
notZeroAddress(account) notPublicHandle(handle) onlyAllowed(handle) {
+ function allow(bytes32 handle, address account) external override
notZeroAddress(account) notPublicHandle(handle) onlyPersistentAdmin(handle) {
      _getNoxComputeStorage().admins[handle][account] = true;
      emit Allowed(msg.sender, handle, account);
  }
+ function disallow(bytes32 handle, address account) external
notPublicHandle(handle) onlyPersistentAdmin(handle) {
+     _getNoxComputeStorage().admins[handle][account] = false;
+     emit Disallowed(msg.sender, handle, account);
+ }
```

---

80 3. Viewer rights are permanent and reachable from transient access

`NoxCompute.addViewer` · Confidence: 80

Description

`addViewer` is gated by `onlyAllowed` (so transient holders qualify) and there is no `removeViewer` — directly contradicting the in-source TODO ("Make viewer expirable") and granting any party that ever holds transient access the ability to permanently authorize off-chain decryption for a third party.

Fix

```
- function addViewer(bytes32 handle, address viewer) external override
notZeroAddress(viewer) notPublicHandle(handle) onlyAllowed(handle) {
+ function addViewer(bytes32 handle, address viewer) external override
notZeroAddress(viewer) notPublicHandle(handle) onlyPersistentAdmin(handle) {
      _getNoxComputeStorage().viewers[handle][viewer] = true;
      emit ViewerAdded(msg.sender, handle, viewer);
  }
+ function removeViewer(bytes32 handle, address viewer) external
notPublicHandle(handle) onlyPersistentAdmin(handle) {
+     _getNoxComputeStorage().viewers[handle][viewer] = false;
+     emit ViewerRemoved(msg.sender, handle, viewer);
+ }
```

---

80 4. SDK returns `address(0)` for Arbitrum mainnet, bricking integrators

`Nox.noxComputeContract` · Confidence: 80

Description

The mainnet branch hardcodes `address(0)` with a `// TODO: Update after mainnet deployment` — every integrator deployed on chainid 42161 sees all `Nox.*` helpers (`add`, `allow`, `fromExternal`, `publicDecrypt` …) revert at the EXTCODESIZE check (or silently no-op on void calls), bricking the SDK on mainnet rather than failing with a clear "unsupported chain" error.

Fix

```
- if (block.chainid == 42161) {
-     // TODO: Update after mainnet deployment.
-     return address(0);
- }
+ // Arbitrum mainnet (42161) intentionally omitted until the production NoxCompute
address is known.
+ // Falling through to the explicit revert below gives integrators a clear
unsupported-chain error.
  ...
  revert UnsupportedChainID(block.chainid);
```

---

75 5. `disallowTransient` allows cross-party transient revocation (DoS)

`NoxCompute.disallowTransient` · Confidence: 75

Description

`disallowTransient(handle, account)` clears `tstore(keccak(handle, account))` after only an `onlyAllowed(handle)` check; any allowed party (including a transient one) can revoke any other party's transient access mid-tx, enabling griefing of multi-contract orchestration flows that thread transient ACL through callbacks.

---

75 6. Gateway address not set during `initialize`

`NoxCompute.initialize` · Confidence: 75

Description

`initialize` sets owner, kmsPublicKey, and proofExpirationDuration but never `$.gateway`; signature checks in `validateInputProof` / `validateDecryptionProof` then compare recovered signers against `address(0)` until the owner remembers to call `setGateway`. Today's safety relies entirely on OZ ECDSA reverting on zero recovery — flagged by 5 of 8 agents.

---

75 7. `validateDecryptionProof` has no on-chain ACL coupling

`NoxCompute.validateDecryptionProof` · Confidence: 75

Description

The function verifies only a gateway signature over `(handle, decryptedResult)` and never checks `$.isPubliclyDecryptable[handle]` or `HandleUtils.isPublicHandle(handle)`; a single mis-

signed off-chain decryption (gateway bug, key compromise, or replay) leaks plaintext on-chain regardless of the on-chain "publicly decryptable" flag, with no defense-in-depth.

Findings List

#	Confidence	Title
1	[85]	Transient ACL holder can permanently mark handle publicly decryptable
2	[82]	Transient ACL holder can mint persistent admins for third parties
3	[80]	Viewer rights are permanent and reachable from transient access
4	[80]	SDK returns `address(0)` for Arbitrum mainnet, bricking integrators
5	[75]	`disallowTransient` allows cross-party transient revocation (DoS)
6	[75]	Gateway address not set during `initialize`
7	[75]	`validateDecryptionProof` has no on-chain ACL coupling

Leads

Vulnerability trails with concrete code smells where the full exploit path could not be completed in one analysis pass. These are not false positives — they are high-signal leads for manual review. Not scored.

- **`select` does not validate result type is supported** — `NoxCompute.select` — Code smells: no `validateArithmeticType(resultType)` call, types beyond Bool/Uint16/Uint256/Int16/Int256 mint handles whose off-chain TEE behavior is undefined — surface is on-chain ACL/event-stream pollution; needs TEE-side rejection confirmation.
- **`validateInputProof` does not check `attrs` byte** — `NoxCompute.validateInputProof` — Code smells: only chainId (byte 1-4) and type (byte 5) are validated; if gateway ever signs `attrs=0x00`, `_allowTransient` no-ops and `isPublicHandle` paths bypass ACL, magnifying any gateway-side bug.
- **`setProofExpirationDuration` unbounded** — `NoxCompute.setProofExpirationDuration` — Code smells: only `onlyOwner`; setting `0` instantly expires every in-flight proof (DoS of input-proof onboarding), `type(uint256).max` disables expiration; no bounds, no timelock.
- **Single-step ownership on UUPS proxy** — `NoxCompute` — Code smells: inherits `OwnableUpgradeable` not `Ownable2StepUpgradeable`; owner gates `_authorizeUpgrade`, `setGateway`, `setKmsPublicKey`, `setProofExpirationDuration`; one mistyped `transferOwnership` is unrecoverable.
- **`setKmsPublicKey` missing format/length check** — `NoxCompute.setKmsPublicKey` — Code smells: only `length != 0` is enforced despite the documented 33-byte SEC1 compressed secp256k1 format; off-chain encryption clients can silently produce undecryptable ciphertexts.
- **`validateInputProof` replay within expiration window** — `NoxCompute.validateInputProof` — Code smells: no nonce, no consumed-proof marker; the bound app can re-consume the same proof any number of times within the (default 1h) window; on-chain effect is idempotent but off-chain accounting that treats the call as one-shot will miscount.

- **Chain-ID truncated to 32 bits in handles** — `HandleUtils.zeroHandle` / `NoxCompute._generateHandle` / `NoxCompute.validateInputProof` — Code smells: `bytes4(uint32(block.chainid))`; current targets (42161, 421614, 31337) fit but the library has no compile-time guard preventing future use on a chain whose ID exceeds 2^32.
- **`Nox._resolveUndefinedHandle` silently substitutes the public zero handle** — `Nox._resolveUndefinedHandle` — Code smells: any e* handle equal to `bytes32(0)` is rewritten to `HandleUtils.zeroHandle(teeType)` before reaching `_requireDefinedHandles`, so a caller bug that forgets to initialize an encrypted variable computes against zero rather than reverting.
- **`wrapAsPublicHandle` does not validate plaintext fits TEE type** — `NoxCompute.wrapAsPublicHandle` — Code smells: `(value, teeType)` is accepted as-is, so a Bool can wrap `0xFF…FF`, a Uint16 can wrap a value > 2^16-1; on-chain ops accept the resulting handle, off-chain TEE rejects.
- **`wrapAsPublicHandle` calls `_allowTransient` on a public handle (dead code)** — `NoxCompute.wrapAsPublicHandle` — Code smells: `_allowTransient(result, msg.sender)` returns immediately because `isPublicHandle(result)` is true; either remove the call or revisit whether some path should mint a confidential handle here.
- **`HandleUtils.isPublicHandle` only masks bit 0 of byte 6** — `HandleUtils.isPublicHandle` — Code smells: bits 1-7 of `attrs` are unconstrained; ACL keys on full bytes32 so two handles differing only in unused attr bits would be ACL-distinct but `isPublicHandle`-equivalent, brittle for future attribute additions.
- **Nox SDK `addViewer` / `allowPublicDecryption` not consistent with `allow` for public handles** — `Nox.addViewer` / `Nox.allowPublicDecryption` — Code smells: `allow` / `allowTransient` use `_allowIfNotPublic` to silently skip public handles, but `addViewer` / `allowPublicDecryption` call NoxCompute directly and revert via `notPublicHandle`; inconsistent surface area is a footgun for SDK consumers.
- **`disallowTransient` reverts on public handles while `_allowTransient` silently skips** — `NoxCompute.disallowTransient` — Code smells: asymmetric handling between the external function (reverts via `notPublicHandle`) and the internal helper (early-return) can break flows that mix the two without pre-checking publicity.
- **`initializeV2` is permissionless** — `NoxCompute.initializeV2` — Code smells: `reinitializer(2)` with no `onlyOwner`; safe today (only emits events) but any future addition to `_emitZeroHandleSeeds` becomes permissionlessly callable in the upgrade window.
- **Front-run-vulnerable initialization** — `NoxCompute.initialize` — Code smells: standard `initializer`-gated function; if the deployment script does not bundle proxy creation with the `initialize` call atomically, the resulting `initialOwner` (also UUPS upgrader) can be hijacked. Cannot verify from this bundle.